

Haszowanie

Technika efektywnej realizacji struktury danych "słownik":

- wstaw element do słownika;
- odszukaj element w słowniku;
- w niektórych metodach haszowania również:
 - usuń element ze słownika.

Oznaczmy:

- U = zbiór (uniwersum) wszystkich możliwych elementów (najczęściej: nieskończony);
- S = aktualna zawartość zbioru (słownika),
 $S \subseteq U$, $|S| = n$.

Idea metody:

- elementy zbioru S przechowywane w tablicy $A[0..m-1]$ (tablica haszowana), o pustej początkowej zawartości;
- wartość elementu (klucz) służy do obliczenia adresu (indeksu w tablicy), pod którym ma się znajdować element;
- z tablicą skojarzona jest na stałe funkcja haszująca (mieszająca) odwzorowująca elementy w adresy:
$$h : U \rightarrow \{0, \dots, m-1\}.$$

Uwaga:

- posługujemy się określeniem 'klucz' zamiast 'element', mając na uwadze sytuację najczęstszą, gdy w słowniku przechowujemy elementy - struktury z wyróżnionym polem klucza służącym do identyfikacji elementu;
- zbiór U jest zbiorem wszystkich możliwych wartości klucza;
- dalej, pojęcia 'klucz' i 'element' stosujemy wymiennie.

Haszowanie - szczególny przypadek: adresacja bezpośrednia

Klucz, będący liczbą całkowitą, sam jest indeksem właściwego miejsca tablicy:

$$U = \{0, \dots, m-1\}, \quad h(x) = x,$$

czyli

funkcja haszująca jest identycznością.

operacja Insert (x):

$$A[x] := x;$$

operacja Search(x):

jeśli $A[x]$ niepuste (równoważnie: $A[x]=x$);
to sukces (znaleziono);

operacja Delete(x) również łatwa do zrealizowania:

$$A[x] := \text{NULL};$$

Zauważmy analogię z wektorem bitowym jako realizacją słownika:

$$A[x]=1 \Leftrightarrow x \in S$$

Wersja ogólna:

- $|U| \gg m$, *// znacznie większe*
- $|S| < m$,
- adres elementu oblicza się stosując funkcję haszującą $h : U \rightarrow \{0, \dots, m-1\}$

$|U| > m$, więc możliwe są kolizje:

$u, v \in S, u \neq v$, oraz $h(u) = h(v)$.

Wymagają specjalnego postępowania: "obsługa kolizji".

Funkcja haszująca

Ideał, osiągalny tylko w przybliżeniu:

- łatwo obliczalna,
- losowa, tzn. każdy indeks jednakowo prawdopodobny jako wartość funkcji dla losowego klucza, niezależnie od wartości funkcji haszującej dla innych kluczy znajdujących się w S ;
- **formalnie, zakładamy proste haszowanie równomierne:**
 - losujemy klucz $v \in U$;
 - $\Pr\{h(v)=i\} = 1/m, i=0, \dots, m-1$.

Gdy klucz zajmuje wiele słów (typowa sytuacja: klucz jest łańcuchem znakowym),

to funkcja h składa się z dwóch faz:

- A: wstępna transformacja klucza w jedno słowo maszynowe (np. 4 bajty),
- B: właściwa transformacji słowa w indeks tablicy.

Faza A. Realizowana jest np.

- operatorem XOR na fragmentach o długości jednego słowa;
- zwykłym dodawaniem kolejnych fragmentów (np. bajtów lub słów);
- potraktowaniem łańcucha znaków jako wielomianu, cyframi są kody ASCII kolejnych znaków (czyli przy podstawie 128);
- jak poprzednio, przy podstawie będącej liczbą pierwszą bliską wielkości bajtu/znaku;
- itp., wynik modulo wielkość słowa.

Pożądana własność takiej transformacji:

- jako wynik możliwy jest dowolny układ bitów w słowie reprezentującym ten wynik.

Faza B. Słowo przekształcane na indeks tablicy.

Niech

w - długość słowa maszynowego w bitach,

k = wartość otrzymana w fazie A, $0 < k < 2^w - 1$,

Dwie podstawowe metody:

1. Metoda dzielenia: $h(k) := k \bmod m$;

niewskazane wartości m :

$m = 2^p$, dowolna potęga dwójki,

zalecane:

m = liczba pierwsza odległa od potęgi dwójki.

2. Metoda mnożenia, szczególnie zalecana gdy m jest potęgą dwójki (ale m może być dowolne)

- $m = 2^p$, $p < w$, w - długość słowa,
- A - stała, $0 < A < 1$, A zajmuje całe słowo,
 - czyli $A = s/2^w$, gdzie s - liczba w -bitowa,
 - sugestia (D.Knuth): $A \approx (\sqrt{5} - 1)/2$;
- $h(k) := \lfloor m(kA - \lfloor kA \rfloor) \rfloor$

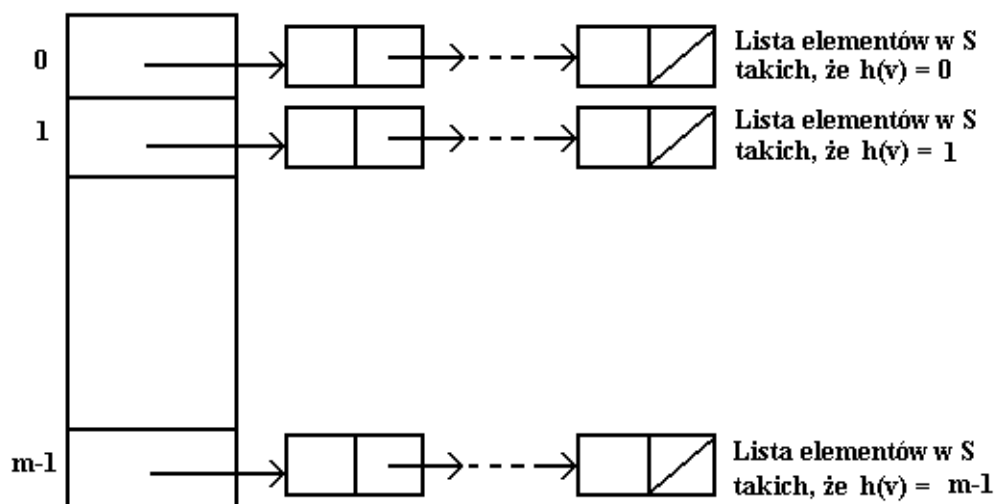
Technicznie, obliczanie $h(k)$:

- $x := 2^w \cdot A \cdot k$ (czyli $x := s \cdot k$)
- x jest $2w$ -bitową liczbę postaci $r_1 \cdot 2^w + r_2$;
- $h(k) := p$ najbardziej znaczących bitów liczby r_2 .

Metody rozwiązywania problemu kolizji

Łańcuchowanie

- $A[i]$ jest nagłówkiem listy zawierającej wszystkie elementy v zbioru S takie, że $h(v) = i$;
- listy zajmują dodatkową pamięć poza tablicą A ;
- zakładamy $n = O(m)$, niekoniecznie $|S| < m$.



Notacja: $\alpha = n/m$ - współczynnik zapęłnienia tablicy A.

Założenie: proste haszowanie równomierne,

- a zatem $\alpha = O(1)$ jest oczekiwaną (średnią) długością listy.

Wyszukiwanie

`Search(v)` : przeglądaj listę `A[h(v)]`.

Złożoność średnia:

- w przypadku sukcesu ($v \in S$): $\Theta(\alpha)$
(połowa oczekiwanej długości listy),
- w przypadku porażki ($v \notin S$): $\Theta(\alpha)$
(oczekiwana długości listy).

Wstawianie

`Insert(v)` : wstaw `v` na koniec listy `A[h(v)]`,
sprawdzając po drodze czy $v \in S$.

Usuwanie

`Delete(v)` : wyszukaj `v` na liście `A[h(v)]` i usuń;

Złożoność średnia `Insert` i `Delete`: jak dla wyszukiwania z porażką, $\Theta(\alpha)$.

Złożoność PESYMISTYCZNA `Search`, `Insert`, `Delete`: $\Theta(n)$

- gdy $h(v)$ jednakowe dla wszystkich kluczy w S .

Wstawianie - wersja uproszczona, rzadziej stosowana

`InsertToFront(v)` : wstaw `v` na początek listy `A[h(v)]`.

Złożoność: $O(1)$.

Kompromis czas – pamięć:

dużo pamięci (duże m) $\Rightarrow$ mała złożoność średnia.

2. Adresowanie otwarte

Założenie: $n < m$, i wszystkie elementy, również kolidujące, umieszczane w tablicy.

Formalnie, haszowanie wyznacza ciąg prób, czyli:

ciąg indeksów, w których kolejno sprawdza się zawartość tablicy:

$$h : U \times \{0, \dots, m-1\} \rightarrow \{0, \dots, m-1\}.$$

Dla poprawności wymagamy, aby ciąg prób

$$h(k, 0), h(k, 1), \dots, h(k, m-1)$$

był permutacją liczb $\{0, 1, \dots, m-1\}$.

Żadna z technik stosowanych praktycznie nie daje potencjalnej możliwości wykorzystania wszystkich $m!$ permutacji jako ciągów prób.

Wybrane techniki adresowania otwartego:

2a. adresowanie (próbkiwanie) liniowe

Funkcja haszująca podstawowa $h' : U \rightarrow 0..m-1$
(jednoargumentowa, tzn. klucz $\rightarrow$ indeks)

Definiujemy ciąg prób:

$$h(k, i) = (h'(k) + i) \bmod m, i=0, 1, \dots, m-1.$$

Czyli jeśli miejsce zajęte, to szukaj pierwszego wolnego, przeglądając kolejne elementy tablicy.

Przykładowa realizacja (*Sedgewick, Algorytmy C++*):

```

Item *ht;          // tablica haszowana, do zaalokowania
int n, m;          // m musi być ustalone przez użyciem
Item nullItem;     // element pusty (zależny od typu Item)
void create() {     // utworzenie tablicy pustej
    ht = new Item[m];
    for (int i=0; i<m; i++) ht[i]=nullItem;
    n = 0;
}
void insert(Item item) { // założenie: n<m
    int j = hash(item.key, m);
    while (ht[j] != nullItem)
        j=(j+1)%m; // możliwy przegląd całej tablicy
    ht[j] = item; n++;
}
Item search (Key v) { // szukaj elementu o kluczu v
    int j = hash(v);
    // kolejne elementy, aż znajdziesz v lub puste
    while (ht[j] != nullItem)
        if (v == ht[j].key) return ht[j];
        else j=(j+1)%m;
    return nullItem;
}

```

Niech $\alpha := n/m$ współczynnik zapełnienia, $\alpha < 1$

Fakt:

Oczekiwana liczba porównań (miejsc przeglądanych):

- wyszukiwanie trafione (czyli z sukcesem):

$$0.5 (1 + 1/(1-\alpha))$$
- wyszukiwanie chybione (czyli z porażką), i tak samo dla wstawiania:

$$0.5 (1 + 1/(1-\alpha)^2)$$

(dowód pomijamy)

Oczekiwana liczba porównań:

$\alpha =$	1/2	2/3	3/4	9/10
sukces	1.5	2.0	2.5	5.5
porażka	2.5	5.0	8.5	50.5

Wniosek: metoda zalecana gdy $\alpha < 1/2$.

Ze wzrostem zapelnienia powstają skupiska miejsc zapelnionych, opóźniające przeszukiwanie.

2b. haszowanie podwójne

- $h(k, i) := (h'(k) + i g(k)) \bmod m$,
czyli podobnie jak przy adresowaniu liniowym,
- ale podczas przeszukiwania w każdej iteracji zwiększamy indeks nie o 1, lecz o wartość $d=g(k)$, gdzie $g()$ jest drugą funkcją haszującą.

0	22
1	55
2	
3	
4	28
5	
6	17
7	26
8	
9	
10	

$m = 11$

$h(k) = k \bmod 11$

$g(k) = 1 + k \bmod 10$

wstaw 17, 22, 28, 26, 55

$h(17) = 6$

$h(22) = 0$

$h(28) = 6$

$h(26) = 4$

$h(55) = 0$

$g(28) = 9$

$(6 + 9) \bmod 11 = 4$

$g(26) = 7$

$(4 + 7) \bmod 11 = 0$

$(0 + 7) \bmod 11 = 7$

$g(55) = 6$

$(0 + 6) \bmod 11 = 6$

$(6 + 6) \bmod 11 = 1$

Założenia:

- $g(v) > 0$, dla każdego v ;
- podczas jednego przeszukiwania może być przeglądana cała tablica, zatem:
 - $\text{NWD}(g(v), m) = 1$, względnie pierwsze;

Praktyczne metody spełnienia tej własności:

- m - liczba pierwsza, albo
- m - potęga 2, $g(v)$ nieparzysta.

Funkcja g powinna być "istotnie różna" od h , np. dla funkcji

$$h(v) = v \bmod m$$

dobrze wyniki daje: $g(v) = m - 2 - v \bmod (m-2)$

Złożoność pesymistyczna: liniowa.

Złożoność średnia - model teoretyczny:

- założenie: haszowanie równomierne, czyli
 - gdy losuję element $v \in U$, każda permutacja prób jednakowo prawdopodobna;
- wynik:
 - wyszukiwanie z sukcesem: $(1/\alpha) \ln(1/(1-\alpha))$;
 - wyszukiwanie z porażką: $1/(1-\alpha)$.

Dowód pomijamy.

Zauważmy

- metoda wtórnej funkcji haszującej wykorzystuje co najwyżej $n(n-1)$ permutacji (ciągów prób);
- metoda adresowania liniowego wykorzystuje co najwyżej n permutacji;

a zatem nie spełniają założenia o haszowaniu równomiernym.

Fakt

Doświadczalna średnia liczba porównań dla metody podwójnego haszowania jest bliska oszacowaniu dla modelu teoretycznego.

- zatem haszowanie podwójne jest praktycznie bardzo efektywne.

Fakt

Adresowanie otwarte nie pozwala efektywnie zrealizować operacji Delete().

Haszowanie kukułcze (cuckoo hashing)

Efektywna realizacja wszystkich operacji słownika:
Search, Insert, Delete.

Tablice haszowane: $A[m]$, $B[m]$, $m \geq 2n$, $n = |S|$.

Dwie różne funkcje haszujące:

$$h_A, h_B : U \rightarrow 0..m-1$$

Funkcja h_A adresuje w tablicy A , h_B w tablicy B .

Niezmiennik:

$$x \in S \Leftrightarrow A[h_A(x)] = x \text{ lub } B[h_B(x)] = x,$$

czyli: x ma dokładnie dwa miejsca, w których może wystąpić, po jednym w każdej tablicy.

Search(x), Delete(x) – oczywiste, $O(1)$.

Insert(x):

- jeśli miejsce $h_A(x)$ w A jest wolne to wstaw do niego x i zakończ;
- jeśli zajęte przez y , to wyrzuć y z tego miejsca i wstaw tam x ;
- w taki sam sposób umieść y w B korzystając z funkcji h_B ;
- jeśli wyznaczone w ten sposób nowe miejsce dla y w B było do tej pory zajęte przez y' , to teraz umieść y' w A korzystając z h_A ;
- itd. na zmianę stosując h_A w A i h_B w B .

W sumie nie więcej razy niż ustalona liczba MAXLOOP.
Jeśli się nie udało, to

- rehash(): zmień funkcje haszujące i wstaw wszystkie elementy dotychczas w A i B (w tym x), od nowa,
- powtarzaj rehash() do skutku.

```
procedure Insert(x)
  if A[hA(x)]=x or B[hB(x)]=x then return;
  for i=1 to MAX_LOOP do
    // MAX_LOOP dobrane jak poniżej
    swap(x,A[hA(x)]);
    if x=NULL then return;
    swap(x,B[hB(x)]);
    if x=NULL then return;
  rehash (x);
end.
```

Założenie o funkcjach haszujących:

aby wykonać rehash() losujemy je z odpowiedniego, uniwersalnego zbioru funkcji haszujących (definicja i własności innym razem).

Fakt

Przy powyższym założeniu o funkcjach haszujących, jeśli $\text{MAX_LOOP} = 6 \lg n$, to oczekiwany koszt operacji Insert() jest stały. (dowód pomijamy)

Uwaga

Inna wersja haszowania kukułczego:

- jedna tablica,
- dwie różne funkcje haszujące,
- algorytmy analogiczne, złożoność również.